

Program Planning Document – COS125 Recursion Assignment.

General Rules & Academic Integrity

- **Concepts Used:** Only use concepts presented in the course to-date. Using advanced concepts or concepts from outside the course does not demonstrate understanding of the required material and may result in a zero for the assignment.
- **Academic Integrity:** *Do not use AI.* Asking AI, online searches, or people not associated with the course is not allowed and leads to a grade of zero and an integrity report. Specify any collaboration you had within the comment header. If you went to Boardman 138, state that.
- **Planning:** Start early. Create a written plan of the processes, data flow, logic, and inputs/outputs required, paying special attention to the nested recursion structure.
- **INCLUSION OF FILES** not related to the assignment will result in grade reduction of **50%**.

Planning Requirements

Before writing any code for your recursive assignment, you must complete and submit a detailed planning document. This document serves as your conceptual blueprint for the program's structure and logic, ensuring your approach meets the strict recursion-only constraint.

Submission: Word Document or PDF. Images from whiteboard drawings may be embedded in the document.

Goal: Provide a comprehensive, high-level written plan that demonstrates you have analyzed the assignment and designed a robust, modular, and recursive solution.

Section 1: Data Flow and Structure Analysis

- **Input and Output Requirements**

State the specific name of the input data file and the output log file.

Describe the overall process of reading the input file and writing to the output file, including any necessary file handling precautions (see Section 3).

- **Raw Data to Python Structure Conversion**

Define the ideal Python data structure (e.g., list of dictionaries, nested dictionaries) that accurately represents the three-level hierarchy of the input file.

Level 1 (Top Container): Describe the unique string marker that identifies the start of the top-level unit. Explain how your parser will use this marker to logically group the entire collection of data.

Level 2 (Middle Container): Describe the conceptual challenge of determining data separation within the ongoing stream of raw input. What is the logic required to identify a new grouping and assign subsequent data lines to that group within the Level 1 structure?

Level 3 (Leaf Node): Explain the conversion logic: How will the basic comma-separated value pair (e.g., 1000,95.50) be parsed, type-converted, and structured (e.g., into a dictionary with specific keys) to prepare it for mathematical analysis?

Section 2: Recursive Function Design and Logic

Your solution must be modular, with each primary task handled by a dedicated recursive function. Do not use loops.

- **Functional Modules**

Describe the purpose, expected data input, and expected data output for each of these primary functions:

- **Parsing/Structure Building:** Converts raw input lines into the final nested data structure.
- **Max/Min Analysis:** Finds the required peak values for the target subsystem across a given data set.
- **Segment Filtering:** Isolates specific subsets of the main data structure (e.g., training segments).

Detailed Recursive Logic Prompt

For the core recursive functions (Parsing and Max/Min Analysis), use the following prompts to guide your design description:

- **Base Case Definition:** What is the simplest, non-recursive condition that causes the function to stop calling itself and return a result? (e.g., a list is empty, a specific count is reached).
- **Recursive Step Mechanics:**
 - What piece of data is processed in the current function call?
 - How is the remaining data set modified (reduced) before the function calls itself again?
 - **Nested Recursion (For Max/Min Analysis):** Since the data is nested, explain the handoff between your functions:
 - **Outer Function's Role:** Which level of the hierarchy does it iterate over (Level 1 or Level 2)? What is its responsibility (e.g., comparing results from sub-calls)?
 - **Inner Function's Role:** Which level does it iterate over (Level 2 or Level 3)? What is the single final value it returns to the Outer Function?

Section 3: Program Execution and Control Flow

This section ensures you have planned how the program runs from start to finish to meet all assignment requirements.

3.1. Initial Setup and File Handling

- Describe the error-handling logic you will implement to protect existing output files. (How will your program respond if the output file already exists, as required by the assignment?)

3.2. The Program's "Air Traffic Control"

- **Design Concept:** Describe the central block of code responsible for orchestrating the entire program.
- Explain its responsibilities in managing the flow:
 - Its first job (initial setup, input collection).
 - Its role in sequentially calling the specialized recursive functions (Parsing, Analysis, Filtering).
 - Its final responsibility (gathering all results and initiating the reporting function).
 - Explain how it ensures that the output from one function (e.g., the filtered data) is correctly passed as the input to the next function (e.g., the analysis function).

3.3. Overall Program Sequence

Outline the complete, sequential flow of execution needed to calculate all four required values (Overall Max/Min and Training Max/Min):

- Prompt for User Input.
- Read File and Call Function A (Parsing).
- Call Function B (Analysis) on Full Data.
- Call Function C (Filtering).
- Call Function B (Analysis) on Filtered Data.
- Write Final Report.

3.4. Report Generation

List the seven specific pieces of data (all four calculated values plus file name, subsystem, and total pulse count) that must be assembled and written clearly to the final output log file.